Retinal OCT with Calibrated Confidence

Project 1 · Medical imaging build guide · exported 23 Oct 2026

You'll build a classifier for retinal OCT scans (CNV, DME, drusen, normal) and then test whether its **confidence can be trusted**: are the scans it is unsure about really the ones it gets wrong? That question is the bridge between this project and your caption research, where you show ASR confidence to Deaf and hard-of-hearing viewers.

Pace: 11 steps over about two weeks at 1–2 hours a day. Each step tells you *why* it exists, *what* to do, *what to learn first*, *what you get* from it, *alternatives* you could take instead, and *how you know it's done*.

Rules for yourself

- **Work in order.** Every step uses what the previous one produced.
- **Write the code yourself.** Read documentation and papers, not finished notebooks. A professor can tell in minutes whether you understand your own repo.
- **Commit after every step** with a message that says what changed. Your history shows how you work.
- **Keep `NOTES.md`:** one line a day on what you tried, what broke and what you learned. Step 11 turns it into your write-up.

When something is unclear: select the confusing words anywhere on this page and press the ✦ button. A more detailed version opens in a panel on the right. You can expand that version again, and choose which version this guide shows.

01 Set up your workspace

Day 1 · about 1 hour

WHY THIS STEP

Everything later depends on a setup that survives crashes and that someone else can rerun. Colab gives you a free GPU but deletes its disk when the session ends, and Git is how a professor will judge your work. Ten minutes of setup now saves hours of lost checkpoints later.

WHAT YOU'LL DO

- [x] **Create a public GitHub repo** called `oct-calibration`, with a README, a Python `.gitignore`, and folders `src/`, `notebooks/` and `runs/`. Add `runs/` to `.gitignore` so model files never get committed.
- [x] **Open a Colab notebook with a T4 GPU** (Runtime → Change runtime type) and mount Google Drive. Data and checkpoints go to Drive so a disconnect doesn't wipe them.
- [x] **Create a Kaggle account and API token** (Settings → Create New Token). This downloads `kaggle.json`, which lets Colab fetch the dataset with one command.
- [ ] **Decide your code layout:** real code lives in `.py` files under `src/`; the notebook only clones the repo and calls those files. Write this rule at the top of `NOTES.md`.

LEARN FIRST

- Colab FAQ: GPU limits, why sessions disconnect, and what is kept.
- GitHub Hello World: repos, commits and branches if Git is rusty.
- Kaggle API docs: where `kaggle.json` goes and how `kaggle datasets download` works.

WHAT YOU GET

A notebook that can `git clone` your repo, see a GPU in `nvidia-smi`, and read and write files in Drive. From here on, each work session starts the same way in under two minutes.

ALTERNATIVES

- **Kaggle Notebooks** instead of Colab: about 30 GPU hours a week and the dataset is already attached, so no download. The trade-off is less flexible file storage. Kaggle notebooks docs
- **Your own laptop:** fine for Steps 2–4 (data exploration) on a CPU, but training on CPU would take days.
- **Notebook-only project:** faster to start, but harder to rerun and looks less professional. Avoid it for a portfolio piece.

DONE WHEN

Colab clones your repo, `nvidia-smi` shows a T4, and an empty first commit is on GitHub.

WATCH FOR

- Never commit `kaggle.json`; it's a password. Upload it to Colab each session instead.
- Free Colab GPUs time out after idle periods. Save to Drive often.

02 Get the data and explore it

Days 1–2

WHY THIS STEP

You can't judge a model on data you haven't looked at. Exploring first tells you how imbalanced the classes are, what the images look like, and, most importantly here, that each file carries a **patient ID**, which decides how you must split the data in Step 3.

WHAT YOU'LL DO

- [x] **Download Kermany OCT2017** (Mendeley, or Kaggle `paultimothymooney/kermany2018`, about 5 GB) into Drive and unzip it.
- [ ] **Count images per class in train and test.** Record the counts and the ratio between the largest and smallest class in `NOTES.md`.
- [ ] **Look at 20 images per class** in a grid. Write one line per class on what you see: fluid pockets, bumps under the retina, or a smooth layered retina.
- [ ] **Check image sizes and modes** (grayscale or RGB, width × height). Sizes vary, which matters when you resize in Step 4.
- [ ] **Extract the patient ID from each file name.** Names look like `CNV-1016042-1.jpeg`: class, patient ID, scan number.
- [ ] **Count patients per split, and how many patients appear in both train and test.** Save a sample grid image per class to `runs/`.

LEARN FIRST

- Optical coherence tomography (Wikipedia): how OCT produces a cross-section of the retina.
- Choroidal neovascularization, Macular edema (DME is the diabetic form) and Drusen: what each class means.
- Age-related macular degeneration (EyeWiki): the disease behind CNV and drusen, with OCT images.
- Kermany et al. 2018, *Cell*: skim the methods to learn how the labels were made.

WHAT YOU GET

A short data section in `NOTES.md` with counts, a patient overlap number and visual notes, plus four sample grids. Your README's data section in Step 11 comes straight from this.

ALTERNATIVES

- **OCTMNIST** from MedMNIST: the same images shrunk to 28×28 or larger, one `pip install`. Great for a first pipeline test, but too small to be your main result.
- **Explore with pandas:** build one table with columns `path, class, patient, width, height` and answer every question above with a few `groupby` calls. This is the cleaner way, and the table feeds Step 3 directly.

DONE WHEN

`NOTES.md` has class counts, patient counts per split, the train–test patient overlap number, and one line per class on what the images show.

WATCH FOR

- The training classes are heavily imbalanced (CNV has many times more images than drusen). Remember the ratio for Step 5.
- The Kaggle copy nests folders, and one folder name may end in a space. Print the folder names with `repr()` if a path fails.

03 Split by patient

Day 2

WHY THIS STEP

Scans from the same patient look alike. If one patient's scans land in both training and validation, the model can recognise *the patient* instead of *the disease*, and your accuracy looks better than it really is. This is called **data leakage**, and it's the most common mistake reviewers look for in medical imaging work.

WHAT YOU'LL DO

- [ ] **Write** `src/data.py` with a function that lists every image as `(path, label, patient_id)`.
- [ ] **Split the official training images into train and validation by patient**, about 90/10, with a fixed random seed. Every scan from one patient goes to the same side.
- [ ] **Remove from training any patient who also appears in the official test set**, and record how many you removed.
- [ ] **Add `assert` checks** that no patient ID appears in two splits, and save the final split as a CSV (`path, label, patient, split`).

LEARN FIRST

- scikit-learn `GroupShuffleSplit`: splits so a group (here a patient) never crosses sides.
- Kapoor & Narayanan, *Leakage and the Reproducibility Crisis in ML-based Science*: a catalogue of leakage types, with examples.
- Metrics Reloaded: a widely used guide to evaluating medical image analysis, including how to split data.

WHAT YOU GET

A saved split CSV that every later step reads, so all your results use exactly the same data. The asserts protect you if you change the code later.

ALTERNATIVES

- `GroupKFold` **cross-validation** instead of one validation split: 5 folds give you a mean and spread instead of one number. It costs five times the training, so save it for the final model.
- **Stratified by class *and* grouped by patient** (`StratifiedGroupKFold`): keeps class ratios similar in each split. Worth it if your validation set ends up with very few drusen scans.

DONE WHEN

Your asserts pass, the CSV is saved, and you can explain in one sentence why splitting by image would inflate accuracy.

WATCH FOR

- One patient can have scans in several classes. Split on patient ID only, never on patient plus class.
- If validation accuracy later looks suspiciously close to 100%, come back here first.

04 Build the data pipeline

Day 3

WHY THIS STEP

The model only ever sees what the pipeline gives it. Getting resizing, normalisation and augmentation right here decides whether a pretrained network can use what it already knows, and whether training is fast enough to iterate on.

WHAT YOU'LL DO

- [ ] **Write a PyTorch** `Dataset` that reads one image, applies a transform, and returns `(image, label, index)`. You'll use the index in Steps 8–9 to find which scans the model got wrong.
- [ ] **Convert grayscale to 3 channels and normalise with ImageNet mean and standard deviation.** The pretrained network expects that input.
- [ ] **Choose training augmentations:** small rotation (about ±10°), horizontal flip, a slight random crop, and brightness or contrast jitter. Validation and test get **resize only**.
- [ ] **Plot one augmented batch** and check every image still looks like a plausible retina scan.

LEARN FIRST

- PyTorch: Datasets and DataLoaders: writing `__len__` and `__getitem__`.
- torchvision transforms: `Resize`, `RandomResizedCrop`, `ColorJitter`, `Normalize`.
- torchvision pretrained models: the input size and normalisation each model expects.

WHAT YOU GET

Three `DataLoader`s (train, validation, test) that load a batch in well under a second. Augmentation gives the model more variety than the raw images, which reduces overfitting.

ALTERNATIVES

- **Albumentations** (docs): faster and richer augmentations. Overkill for this project, but common in medical imaging.
- **Pre-resize all images once** to 256 px and save them locally: loading gets much faster on Colab, at the cost of one extra preprocessing script.
- **Single-channel input:** change the first convolution to accept 1 channel. Cleaner, but you lose part of the pretrained weights' benefit; the 3-channel copy is the usual choice.

DONE WHEN

One batch loads in under a second on Colab and the plotted batch looks right.

WATCH FOR

- **No vertical flips.** An upside-down retina never occurs in real scans.
- **No augmentation on validation or test**, or your results change from run to run.
- If loading is slow, copy the data from Drive to Colab's local disk first and raise `num_workers`.

05 Train a baseline

Days 4–5

WHY THIS STEP

You need one solid, simple model before anything interesting can happen. Calibration (Step 7) and selective prediction (Step 8) are measured on this model's outputs, so it has to be trained carefully and its raw outputs saved.

WHAT YOU'LL DO

- [ ] **Load an ImageNet-pretrained ResNet-18** from torchvision and replace its final layer with a 4-output layer.
- [ ] **Handle class imbalance** with either a `WeightedRandomSampler` or a class-weighted loss, not both. Write down which and why in `NOTES.md`.
- [ ] **Set up training:** AdamW optimiser, a cosine or one-cycle learning-rate schedule, and mixed precision (`torch.autocast`).
- [ ] **Log each epoch:** training loss, validation accuracy and validation **macro-F1**. Save the checkpoint with the best validation macro-F1.
- [ ] **Run a quick test first:** 3 epochs on 10,000 training images, to check the whole loop works. Then the full run, about 8 epochs.
- [ ] **Save raw logits** (the outputs before softmax) for validation and test, with labels and file paths. Steps 6–8 only read this file, so you never retrain to try an idea.

LEARN FIRST

- PyTorch transfer learning tutorial: replacing the head and fine-tuning.
- Karpathy, *A Recipe for Training Neural Networks*: how to debug training step by step. Read it before your first run.
- scikit-learn `f1_score`: what `average='macro'` means and why it beats accuracy with imbalanced classes.
- PyTorch automatic mixed precision and `WeightedRandomSampler`.

WHAT YOU GET

A trained checkpoint, a training log you can plot, and a logits file. Published results on this dataset are high, so expect validation macro-F1 above about 0.90. A much lower score means a bug, not a hard dataset.

ALTERNATIVES

- **EfficientNet-B0 or ConvNeXt-Tiny** (via timm): often a little more accurate. Keep ResNet-18 as the baseline either way, because it's the standard comparison.
- **Focal loss** (Lin et al.) for imbalance: it changes how confident the model is, which interacts with calibration. Interesting for Step 10, risky for a baseline.
- **Label smoothing:** improves calibration on its own (Müller et al.), so it would blur your Step 7 comparison. Leave it off for now.
- **Experiment tracking:** TensorBoard or Weights & Biases instead of printing to the console.

DONE WHEN

Validation macro-F1 is above about 0.90, the best checkpoint is in Drive, and the logits file exists.

WATCH FOR

- Near-perfect accuracy in the first epoch usually means leakage. Go back to Step 3.
- Save checkpoints to Drive, not Colab's disk.
- Fix and record the random seed.

06 Evaluate properly

Day 6

WHY THIS STEP

One accuracy number hides which diseases the model confuses and how much the number would move on a different test set. Per-class results and confidence intervals are what make a result believable to a reviewer.

WHAT YOU'LL DO

- [ ] **On the test logits, compute** accuracy, macro-F1, per-class precision, recall and F1, and one-vs-rest AUC.
- [ ] **Plot a confusion matrix** and write down which two classes are confused most, and a guess why from what you saw in Step 2.
- [ ] **Add 95% bootstrap confidence intervals** for accuracy and macro-F1: resample the test set with replacement 1,000 times and take the 2.5th and 97.5th percentiles.

LEARN FIRST

- scikit-learn model evaluation guide: `classification_report`, `confusion_matrix`, `roc_auc_score(multi_class='ovr')`.
- Bootstrapping (Wikipedia): why resampling the test set estimates uncertainty in a metric.

WHAT YOU GET

A results table with intervals and one sentence on the main confusion. This becomes the core table of your README.

ALTERNATIVES

- **Patient-level bootstrap:** resample patients instead of images, so scans from one patient move together. More correct for medical data, slightly more code.
- **Evaluate on a second dataset** (for example OCTMNIST's test split) to see how well the model transfers. A strong addition if time allows.

DONE WHEN

A results table with confidence intervals, a saved confusion matrix figure, and one sentence explaining the main confusion.

WATCH FOR

- The official test set is small and balanced, so intervals may be wider than you expect. Report them anyway; that's the honest result.
- Never tune anything on the test set. Every choice is made on validation.

07 Measure and fix calibration

Days 7–8

WHY THIS STEP

A model is **calibrated** if, among scans where it says "90% sure", it is right about 90% of the time. Modern neural networks are usually over-confident. If the confidence numbers are wrong, nobody can use them to decide when to trust the model, which is the whole point of this project and of your caption research.

WHAT YOU'LL DO

- [ ] **Turn logits into probabilities** with softmax. Confidence is the highest probability for each scan.
- [ ] **Implement expected calibration error (ECE) yourself** with 15 equal-width bins: in each bin, compare average confidence with accuracy, then average the gaps weighted by bin size.
- [ ] **Draw a reliability diagram:** confidence on the x-axis, accuracy per bin on the y-axis, with the diagonal for perfect calibration.
- [ ] **Implement temperature scaling:** learn one number T that divides all logits, fitted on **validation** logits by minimising negative log-likelihood.
- [ ] **Apply T to the test logits** and recompute ECE and the diagram. Confirm accuracy is unchanged.

LEARN FIRST

- Guo et al. 2017, *On Calibration of Modern Neural Networks*: read sections 2 and 4.2. It defines ECE, reliability diagrams and temperature scaling.
- scikit-learn probability calibration guide: a gentle visual introduction to calibration curves.

WHAT YOU GET

Before and after reliability diagrams side by side, the ECE values and the fitted T. Most likely T comes out above 1, meaning the model was over-confident and scaling softens it.

ALTERNATIVES

- **Adaptive (equal-mass) bins** for ECE: each bin holds the same number of scans, which is more stable when most predictions are very confident.
- **Vector or matrix scaling:** one temperature per class instead of one overall. More flexible, but easier to overfit a small validation set.
- **Report Brier score and NLL** alongside ECE: they measure calibration and accuracy together and don't depend on bin choices.

DONE WHEN

You have both diagrams, both ECE values and T, plus a note in `NOTES.md` saying whether the model was over- or under-confident and how the diagram shows it.

WATCH FOR

- Fitting T on the test set is cheating. Always use validation.
- If accuracy changes after scaling, there's a bug: dividing by a positive T never changes which class is highest.

08 Selective prediction

Day 9

WHY THIS STEP

This step answers the project's real question: **does low confidence point at the mistakes?** If it does, a system can answer the scans it is sure about and send the rest to an ophthalmologist. It's the same idea as flagging uncertain words in your captions, and it gives you one headline sentence for your README and your emails.

WHAT YOU'LL DO

- [ ] **Sort test scans by calibrated confidence**, highest first.
- [ ] **Plot a risk–coverage curve:** for each coverage (answering the top 10%, 20% … 100% of scans), the error rate among the scans answered. Compute the area under it (AURC).
- [ ] **Choose a confidence threshold on validation** so the model answers 90% of scans, then apply that threshold to test.
- [ ] **Report** accuracy on answered scans, accuracy on flagged scans, and the share of all test errors that fall in the flagged group.

LEARN FIRST

- Geifman & El-Yaniv 2017, *Selective Classification for Deep Neural Networks*: risk–coverage curves and choosing thresholds.
- Your own @ConfidenceSlow paper in Marginalia: the same idea applied to speech recognition errors.

WHAT YOU GET

One sentence with your real numbers: *"If the model answers 90% of scans and flags the rest for an ophthalmologist, it catches X% of its own mistakes, and its accuracy on what it answers rises from A% to B%."* That sentence opens your README.

ALTERNATIVES

- **Other uncertainty scores:** entropy of the probabilities, or the gap between the top two classes. Compare their curves with plain confidence.
- **Pick the threshold by target accuracy** instead of coverage, for example "answer only when expected accuracy is at least 99%". Closer to how a clinic would set it.

DONE WHEN

You can fill in the outcome sentence above with real numbers, and the risk–coverage plot is saved.

WATCH FOR

- Choose the threshold on validation, never test.
- A flat risk–coverage curve means confidence doesn't separate right from wrong. That's still a result worth reporting.

09 Look inside with Grad-CAM

Day 10

WHY THIS STEP

Numbers say *how often* the model is wrong; heatmaps hint at *why*. If confident mistakes focus on image borders or text instead of the retina, the model has learned a shortcut. Clinicians and reviewers expect this kind of check.

WHAT YOU'LL DO

- [ ] **Implement Grad-CAM yourself** with forward and backward hooks on the last ResNet block (`layer4`). It's about 25 lines once you understand the paper's section 3.
- [ ] **Make three panels of 6 scans each:** confident and correct, confident but wrong, and least confident.
- [ ] **Write two sentences per panel** in `NOTES.md`: is the heat on the retinal layers and the lesion, or on artefacts?

LEARN FIRST

- Selvaraju et al. 2017, *Grad-CAM*, section 3.
- PyTorch module hooks: `register_forward_hook` and `register_full_backward_hook`.

WHAT YOU GET

Three saved figures and a named pattern shared by the confident mistakes, for example "they all have heavy speckle noise" or "heat sits on the image edge".

ALTERNATIVES

- **The `pytorch-grad-cam` library** (GitHub): use it to check your own implementation, not instead of it.
- **Grad-CAM++ or Score-CAM:** sharper maps, same idea. Optional.
- **Occlusion maps:** slide a grey patch over the image and watch the confidence drop. Slower, but needs no gradients and is easy to explain.

DONE WHEN

Three panels are saved and you can name one pattern the confident mistakes share.

WATCH FOR

Grad-CAM shows where the model looked, not why it was right. Don't claim more than that in the write-up.

10 Add one extension

Days 11–12

WHY THIS STEP

Temperature scaling is the standard baseline. Comparing it with one stronger uncertainty method turns a tutorial-style project into a small piece of research, and gives you something specific to discuss with a professor.

WHAT YOU'LL DO

- [ ] **Pick one option** from the alternatives below and write down why in `NOTES.md`.
- [ ] **Run it and add it to a comparison table:** baseline, temperature scaling and your extension, with columns for accuracy, ECE and AURC.

LEARN FIRST

- Lakshminarayanan et al. 2017, *Deep Ensembles*
- Gal & Ghahramani 2016, *Dropout as a Bayesian Approximation*

WHAT YOU GET

A table showing whether the more expensive method beats one temperature number. Either answer is interesting: "ensembles help a lot" and "temperature scaling is enough here" are both findings.

ALTERNATIVES

- **Deep ensemble (recommended):** 3–5 models with different seeds, average their probabilities. The strongest option for your uncertainty story; costs 3–5× training time.
- **MC dropout:** keep dropout on at test time, run 20 passes per scan, and use the spread as uncertainty. Cheap to train, slower to predict.
- **A second architecture:** ResNet-50 or EfficientNet-B0, to test whether bigger models are worse calibrated, as Guo et al. found.

DONE WHEN

The comparison table is filled in and you've written two sentences on what it shows.

WATCH FOR

Budget your Colab GPU time: an ensemble of 5 full runs takes most of a week of free-tier time. Use 3 models, or fewer epochs, if needed.

11 Write it up

Days 13–14

WHY THIS STEP

A professor will spend about two minutes on your repo. If the point isn't clear on the first screen, the work behind it doesn't count. The write-up is also what you'll link in every outreach email.

WHAT YOU'LL DO

- [ ] **Write the README in this order:** the one-line summary and your Step 8 sentence; the reliability diagram and risk–coverage curve side by side; the results table with intervals; a short method section (data, patient split, model, calibration); Grad-CAM panels with your reading; limitations; exact commands to reproduce.
- [ ] **Turn `NOTES.md` into a "What I learned" section**, including what broke and how you fixed it.
- [ ] **Put all figures in `figures/`** and check the repo runs from a fresh clone.
- [ ] **Ask a friend to follow your README without help**, and fix whatever confuses them.
- [ ] **Add the repo to your CV** and one line about it to your outreach email template.

LEARN FIRST

- Make a README: what a good README contains.
- Look at the READMEs of nnU-Net and pytorch-grad-cam for how research repos present results.

WHAT YOU GET

A public, reproducible repo whose first screen states a result, shows two figures, and links to the details. It's your first medical imaging portfolio piece.

ALTERNATIVES

- **A 4-page arXiv-style report** (LaTeX on Overleaf): more work, but gives a professor something to read offline.
- **A short blog post or LinkedIn write-up** with the two figures: reaches more people, less formal.

DONE WHEN

Final checklist: splits by patient with asserts · T fitted on validation, test accuracy unchanged · 95% intervals · reliability, risk–coverage and Grad-CAM figures in the README · one extension compared · README opens with the selective-prediction sentence · limitations written · repo runs from a fresh clone.

WATCH FOR

Be honest about limitations: one public dataset, scans from one hospital system, image-level labels and no clinical validation. Reviewers trust a project more when it states these itself.